PROBLEM STATEMENT :

Flight ticket prices can be something hard to guess, today we might see a price, check out the price of the same flight tomorrow, it will be a different story. We might have often heard travellers saying that flight ticket prices are so unpredictable. Here you will be provided with prices of flight tickets for various airlines between the months of March and June of 2019 and between various cities.

Size of training set: 10683 records

***Size of test set: 2671 records ***

FEATURES:

Airline: The name of the airline.

Date_of_Journey: The date of the journey

Source: The source from which the service begins.

Destination: The destination where the service ends.

Route: The route taken by the flight to reach the destination.

Dep_Time: The time when the journey starts from the source.

Arrival_Time: Time of arrival at the destination.

Duration: Total duration of the flight.

Total_Stops: Total stops between the source and destination.

Additional_Info: Additional information about the flight

Price: The price of the ticket

You have to use your skills as a data scientist and build a machine learning model to predict the price of the flight ticket.

- ***Importing require library for performing EDA, Data Wrangling and data cleaning***

In [1]:
```
import pandas as pd # for data wrangling purpose
import numpy as np # Basic computation library
import seaborn as sns # For Visualization
import matplotlib.pyplot as plt # ploting package
import warnings # Filtering warnings
warnings.filterwarnings('ignore')
```

```
C:\Users\Prashant\anaconda3\lib\site-packages\pandas\core\arrays\masked.py:60: UserWarning: Pandas requires version '1.
3.6' or newer of 'bottleneck' (version '1.3.5' currently installed).
  from pandas.core import (
```

In [2]:
```
# Importing Flight Prediction dataset Excel file using pandas
df = pd.read_excel('Data_Train.xlsx')
```

In [3]:
```
df.head()
```

Out[3]:

	Airline	Date_of_Journey	Source	Destination	Route	Dep_Time	Arrival_Time	Duration	Total_Stops	Additional_Info	Price
0	IndiGo	24/03/2019	Banglore	New Delhi	BLR → DEL	22:20	01:10 22 Mar	2h 50m	non-stop	No info	3897
1	Air India	1/05/2019	Kolkata	Banglore	CCU → IXR → BBI → BLR	05:50	13:15	7h 25m	2 stops	No info	7662
2	Jet Airways	9/06/2019	Delhi	Cochin	DEL → LKO → BOM → COK	09:25	04:25 10 Jun	19h	2 stops	No info	13882
3	IndiGo	12/05/2019	Kolkata	Banglore	CCU → NAG → BLR	18:05	23:30	5h 25m	1 stop	No info	6218
4	IndiGo	01/03/2019	Banglore	New Delhi	BLR → NAG → DEL	16:50	21:35	4h 45m	1 stop	No info	13302

Comment :

- ***Training dataset contain 10683 rows and 11 columns.***
- ***Our Target variable is Price. We gone predict flight prices using Various Regression Algorithms.***
- ***Some feature with date and time related columns are mention with object datatype. We gone convert them into datetime datatype format along with going to perform some feature engineering over them to create few new columns of our interest.***

Converting Date and time columns from object type to Datetime type

1. Feature Engineering on Date of Journey Columns

In [4]:
```
# Extracting Day from Date_of_journey column
df['Journey_Day'] = pd.to_datetime(df.Date_of_Journey).dt.day
df.head()

# Extracting Month from Date_of_journey column
df['Journey_Month'] = pd.to_datetime(df.Date_of_Journey).dt.month
df.head()

# Dropping Date_of_journey column
df.drop("Date_of_Journey",axis=1,inplace=True)
```

2. Feature Engineering on 'Duration' Column

In [5]:
```
# Conversion of Duration column from hr & Minutes format to Minutes
df['Duration']=df['Duration'].str.replace('h','*60').str.replace(' ','+').str.replace('m','*1').apply(eval)

# convert this column into a numeric datatypes
df['Duration']=pd.to_numeric(df['Duration'])  #df['Duration']=df['Duration'].astype(int)
```

In [ ]:
```
df.head()
```

In [6]:
```
s1="2h 20m"
eval(s1.replace("h","*60").replace(" ","+").replace("m","*1"))
```

Out[6]:
```
140
```

3. Feature Engineering on 'Dep_Time' Column

In [7]:
```
# Extracting Hours from Dep_Time column
df['Dep_Hour']=pd.to_datetime(df['Dep_Time']).dt.hour

# Extracting Minutes from Dep_Time column
df['Dep_Min']=pd.to_datetime(df['Dep_Time']).dt.minute

# Dropping Dep_Time column
df.drop("Dep_Time",axis=1,inplace=True)
```

In [ ]:
```
df.head()
```

4. Feature Engineering on 'Arrival_Time' Column

In [8]:
```
# Extracting Arrival_Hour from Arrival_Time column
df['Arrival_Hour']=pd.to_datetime(df['Arrival_Time']).dt.hour

# Extracting Arrival_Min from Arrival_Time column
df['Arrival_Min']=pd.to_datetime(df['Arrival_Time']).dt.minute

# Dropping Arruval_Time column
df.drop("Arrival_Time",axis=1,inplace=True)
```

In [9]:
```
# print('No of Rows:',df.shape[0])
# print('No of Columns:',df.shape[1])
# pd.set_option('display.max_columns', None) # This will enable us to see truncated columns
df.head()
```

Out[9]:

	Airline	Source	Destination	Route	Duration	Total_Stops	Additional_Info	Price	Journey_Day	Journey_Month	Dep_Hour	Dep_Min
0	IndiGo	Banglore	New Delhi	BLR → DEL	170	non-stop	No info	3897	24	3	22	20
1	Air India	Kolkata	Banglore	CCU → IXR → BBI → BLR	445	2 stops	No info	7662	1	5	5	50
2	Jet Airways	Delhi	Cochin	DEL → LKO → BOM → COK	1140	2 stops	No info	13882	9	6	9	25
3	IndiGo	Kolkata	Banglore	CCU → NAG → BLR	325	1 stop	No info	6218	12	5	18	5
4	IndiGo	Banglore	New Delhi	BLR → NAG → DEL	285	1 stop	No info	13302	1	3	16	50

In [10]:
```
Numerical=df.select_dtypes(exclude="object")
Categorical=df.select_dtypes(include="object")
```

We also need to do some feature engineering on Source and Destination columns, We can see at some place Dehli is mention and other places New Dehli is mention.

First enlist different unique values in categorical variable and afterwards we will perform feature engineering.

In [11]:
```
for i in Categorical:
    print('Unique value counts of ',i, 'Enlisted as Below Table :')
    print('-'*40)
    print(df[i].value_counts())
    print("*"*120)
```

```
Unique value counts of  Airline Enlisted as Below Table :
---------------------------------------
Airline
Jet Airways                          3849
IndiGo                               2053
Air India                            1752
Multiple carriers                    1196
SpiceJet                              818
Vistara                               479
Air Asia                              319
GoAir                                 194
Multiple carriers Premium economy      13
Jet Airways Business                    6
Vistara Premium economy                 3
Trujet                                  1
Name: count, dtype: int64
**************************************************************************************************************************
*
Unique value counts of  Source Enlisted as Below Table :
---------------------------------------
Source
Delhi       4537
Kolkata     2871
Banglore    2197
Mumbai       697
Chennai      381
Name: count, dtype: int64
**************************************************************************************************************************
*
Unique value counts of  Destination Enlisted as Below Table :
---------------------------------------
Destination
Cochin       4537
Banglore     2871
Delhi        1265
New Delhi     932
Hyderabad     697
Kolkata       381
Name: count, dtype: int64
**************************************************************************************************************************
*
Unique value counts of  Route Enlisted as Below Table :
---------------------------------------
Route
DEL → BOM → COK          2376
```

```
BLR → DEL                1552
CCU → BOM → BLR           979
CCU → BLR                 724
BOM → HYD                 621
                         ...
CCU → VTZ → BLR             1
CCU → IXZ → MAA → BLR       1
BOM → COK → MAA → HYD       1
BOM → CCU → HYD             1
BOM → BBI → HYD             1
Name: count, Length: 128, dtype: int64
**************************************************************************************************************
*
Unique value counts of  Total_Stops Enlisted as Below Table :
---------------------------------------
Total_Stops
1 stop      5625
non-stop    3491
2 stops     1520
3 stops       45
4 stops        1
Name: count, dtype: int64
**************************************************************************************************************
*
Unique value counts of  Additional_Info Enlisted as Below Table :
---------------------------------------
Additional_Info
No info                         8345
In-flight meal not included     1982
No check-in baggage included     320
1 Long layover                    19
Change airports                    7
Business class                     4
No Info                            3
1 Short layover                    1
Red-eye flight                     1
2 Long layover                     1
Name: count, dtype: int64
**************************************************************************************************************
*
```

In [8]:
```
"*"*2
```

Out[8]:
```
'**'
```

Observation :

- *"New Delhi" is mention as "Delhi". We we gone regulated it.*
- *No info is mention as 'No Info' few times.*
- *Very Few Premium economy or Business class flight in dataset. This might be due to high ticket price*

In [12]:
```
# Replacing "New Delhi" as "Delhi" in Destination column
df["Destination"] = df["Destination"].replace("New Delhi","Delhi")

# In the column "Additional Info", "No Info" and "No info" are same so replacing it by "No Info
df['Additional_Info'] = df['Additional_Info'].replace("No info", "No Info")
```

Data Integrity Check

Since dataset is large, Let check for any entry which is repeated or duplicated in dataset.

In [13]:
```
df.duplicated().sum()  # This will check the duplicate data for all columns.
```

Out[13]:
```
222
```

Around 222 duplicate data rows. It huge and we gone drop them. There is no point on training model on duplicated data.

In [15]:
```
df.drop_duplicates(keep='last',inplace= True)
```

In [16]:
```
df.shape
```

Out[16]:
```
(10461, 14)
```

Let check if any whitespace, 'NA' or '-' exist in dataset.

In [17]:
```
#df.isin([' ','NA','-','?']).sum().any()
```

Missing value check

In [18]:
```
#Finding what percentage of data is missing from the dataset
df.isnull().sum()
```

Out[18]:
```
Airline           0
Source            0
Destination       0
Route             1
Duration          0
Total_Stops       1
Additional_Info   0
Price             0
Journey_Day       0
Journey_Month     0
Dep_Hour          0
Dep_Min           0
Arrival_Hour      0
Arrival_Min       0
dtype: int64
```

Missing values are present in Total Stops and Route. These variable are categorical in nature, we gone impute them with mode.

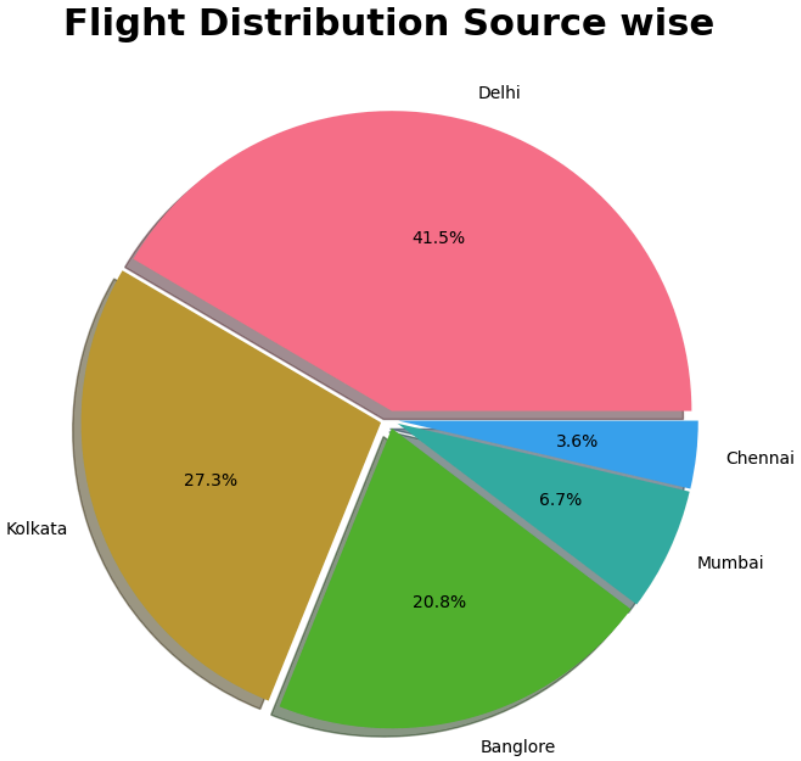
In [19]:
```
# Checking the mode of Categorical columns "Route"
#print("The mode of Route is:",df["Route"].mode())

# Filling the missing values in "Route" withs its mode
df['Route'] = df['Route'].fillna(df['Route'].mode()[0])

# Checking the mode of Categorical columns "Total_Stops"
#print("The mode of Total_Stops is:",df["Total_Stops"].mode())

# Filling the mising values in "Total_Stops" by its mode
df['Total_Stops']=df['Total_Stops'].fillna(df['Total_Stops'].mode()[0])
```

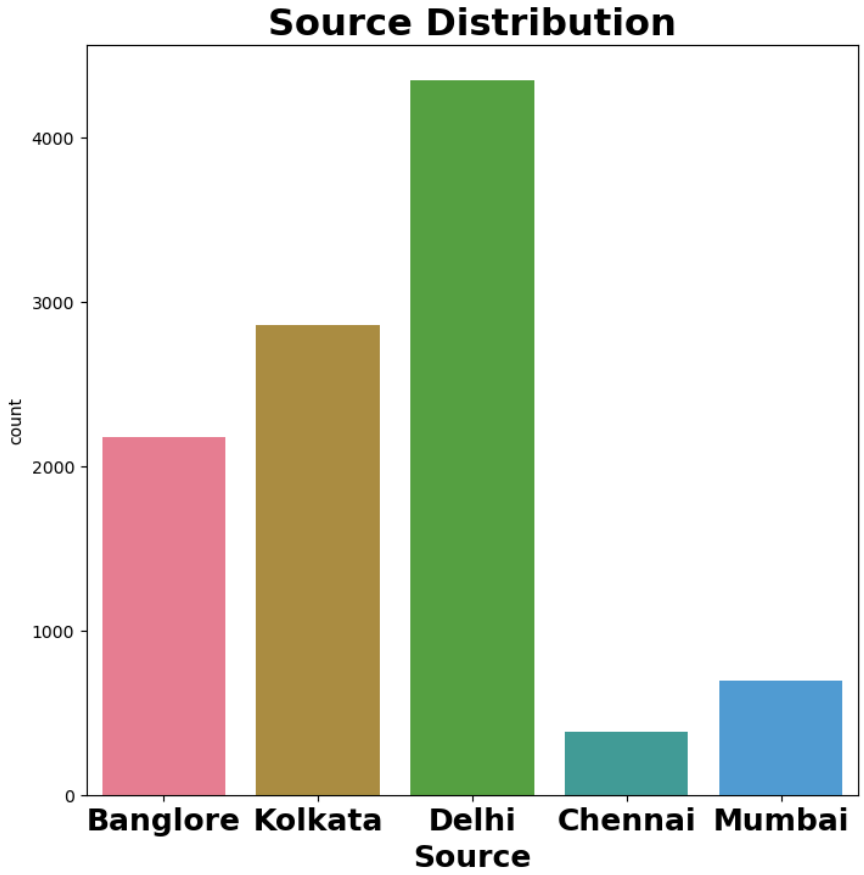
In [ ]:
```
.mode()---- 0   text

.mode()[0]----text
```

Exploring Features Source

In [23]:
```
#plt.rcParams["figure.autolayout"] = True
sns.set_palette('husl')
f,ax=plt.subplots(1,2,figsize=(18,8))
df['Source'].value_counts().plot.pie(explode=[0.03,0.03,0.03,0.03,0.03],autopct='%3.1f%%',
                                     shadow=True,ax=ax[0])
ax[0].set_title('Flight Distribution Source wise', fontsize=22,fontweight ='bold')
```

```
ax[0].set_ylabel('')
sns.countplot(x='Source',data=df,ax=ax[1])
ax[1].set_title('Source Distribution',fontsize=22,fontweight ='bold')
ax[1].set_xlabel("Source",fontsize=18,fontweight ='bold')
plt.xticks(fontsize=18,fontweight ='bold')
plt.show()
```





Observation :

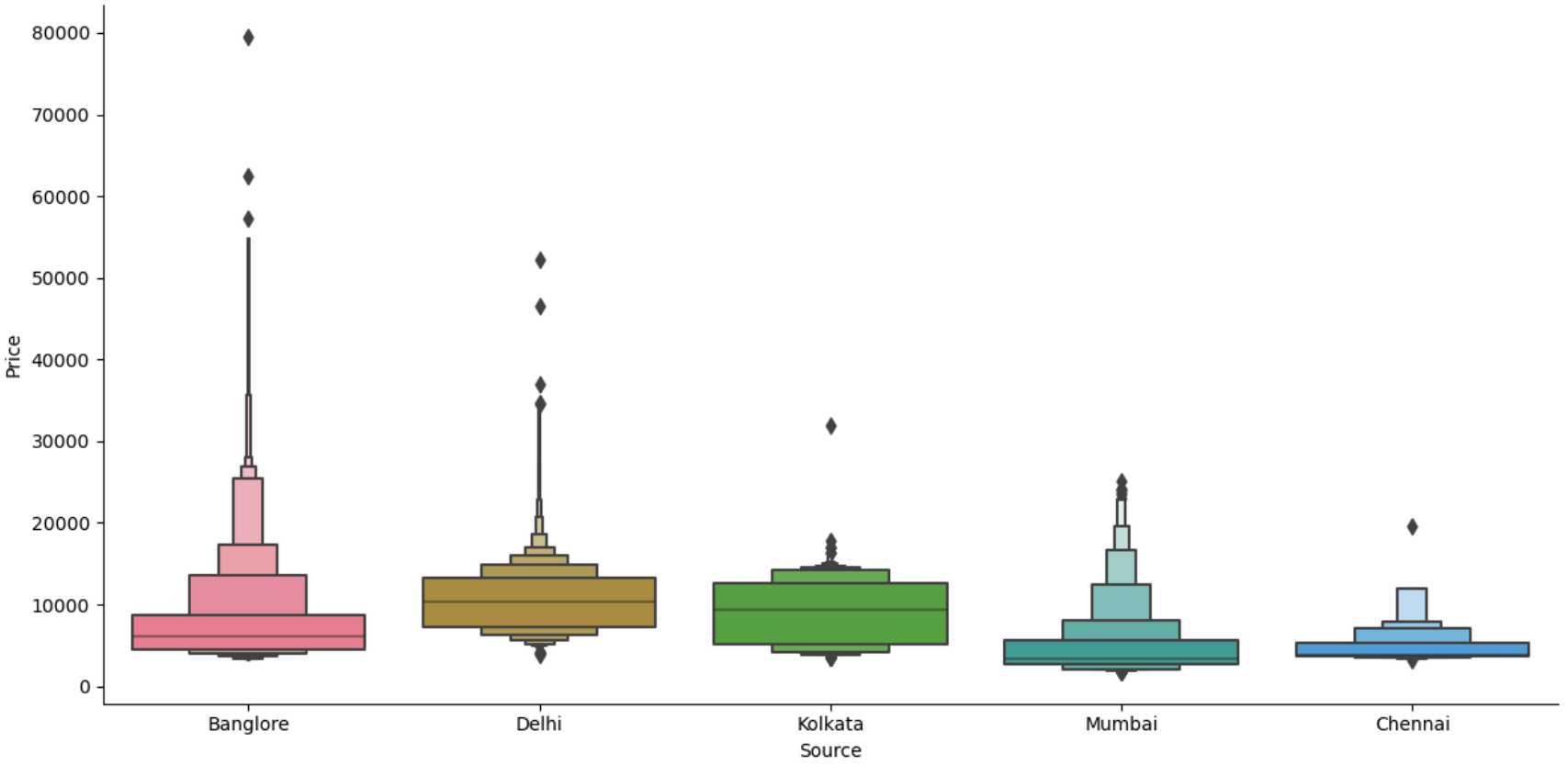
- ***Maximum flight depart from Delhi followed by Kolkata.***

Let Explore Source With respect to Target Variable

In [24]:
```
# Source vs Average Price
plt.figure(figsize=(12,7))
```

```
sns.catplot(y='Price',x='Source',data= df.sort_values('Price',ascending=False),kind="boxen",height=6, aspect=2)
plt.show()
```

```
<Figure size 1200x700 with 0 Axes>
```

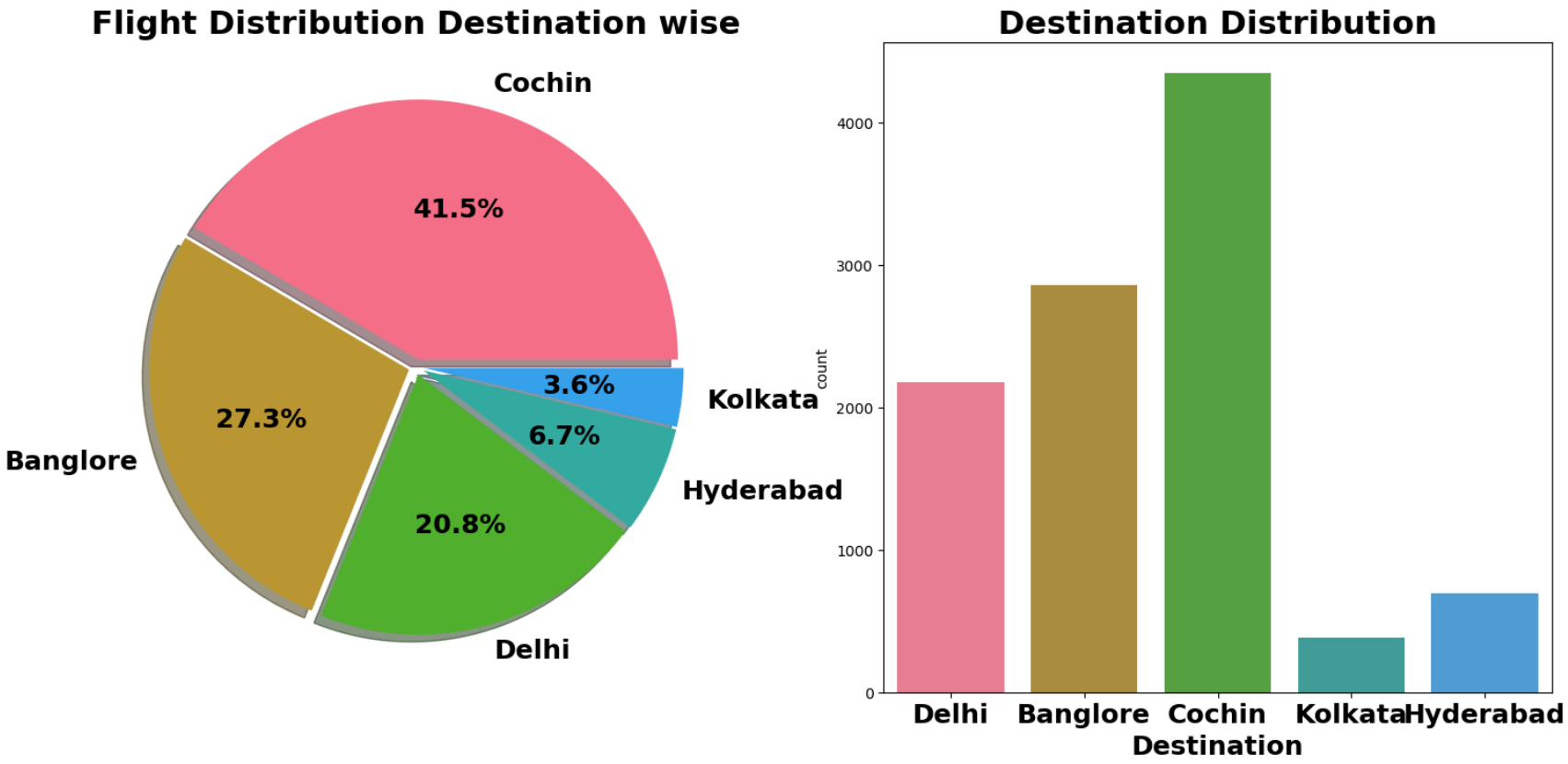


Based upon Source location Maximum Fare Comes for Banglore flight.

Destination VS Price

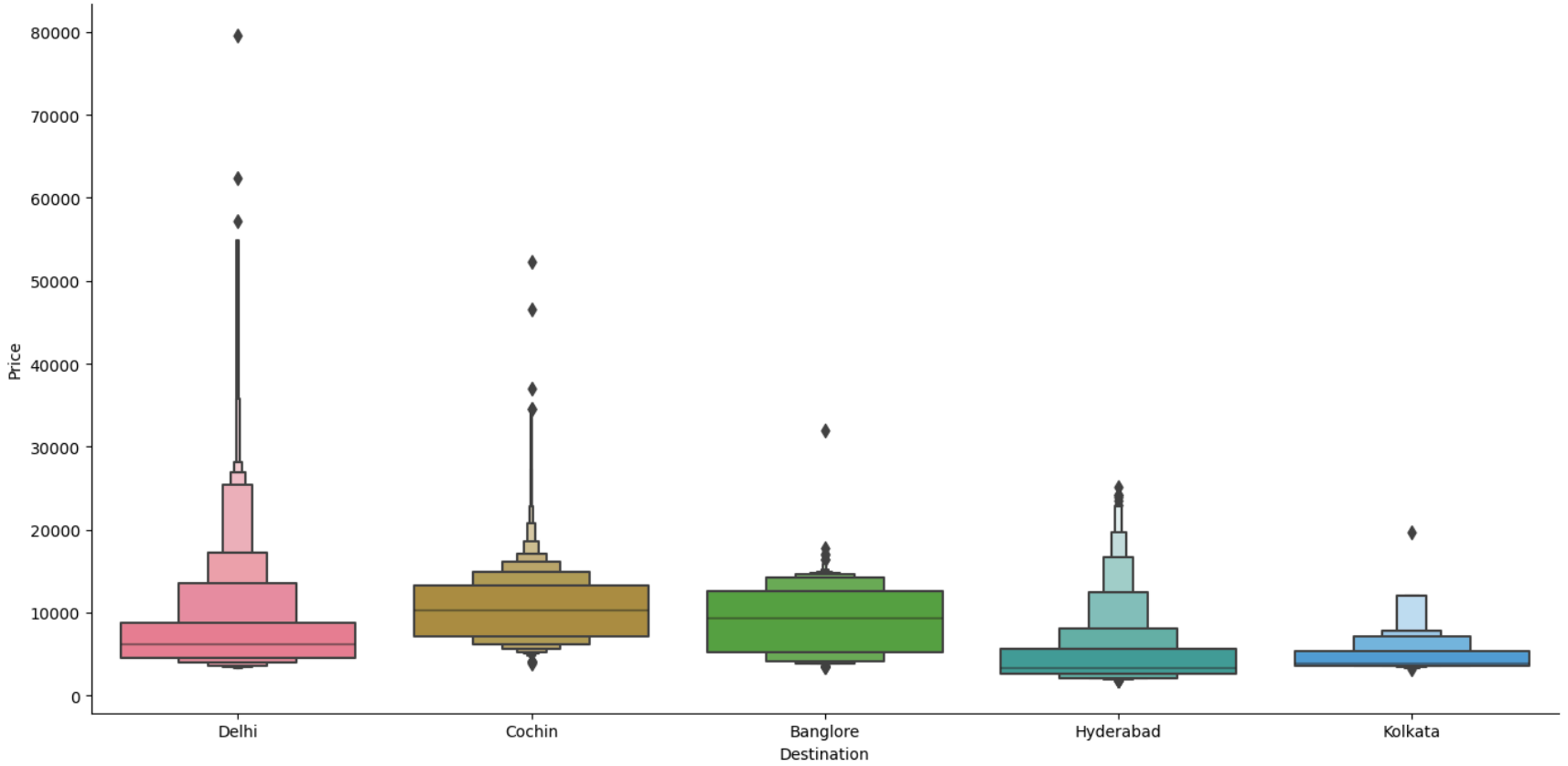
In [25]:
```
#plt.rcParams["figure.autolayout"] = True
sns.set_palette('husl')
f,ax=plt.subplots(1,2,figsize=(18,8))
df['Destination'].value_counts().plot.pie(explode=[0.03,0.03,0.03,0.03,0.03],autopct='%3.1f%%',
                                          textprops ={ 'fontweight': 'bold','fontsize':18}, ax=ax[0],shadow=True)
ax[0].set_title('Flight Distribution Destination wise', fontsize=22,fontweight ='bold')
ax[0].set_ylabel('')
sns.countplot(x='Destination',data=df,ax=ax[1])
```

```
ax[1].set_title('Destination Distribution',fontsize=22,fontweight ='bold')
ax[1].set_xlabel("Destination",fontsize=18,fontweight ='bold')
plt.xticks(fontsize=18,fontweight ='bold')
plt.show()
```



Maximum 41.5% Flight lands into cochin followed by Banglore.

In [26]:
```
# Destination vs AveragePrice
sns.catplot(y='Price',x='Destination',data= df.sort_values('Price',ascending=False),
            kind = "boxen",height = 7, aspect = 2 )
plt.show()
```



The Flight ticket price range in Delhi is the maximum, reason may be traffic & the National Capital, political seat of power and a highly visited place for vacations(same for bangalore & cochin)

Airlines VS Source

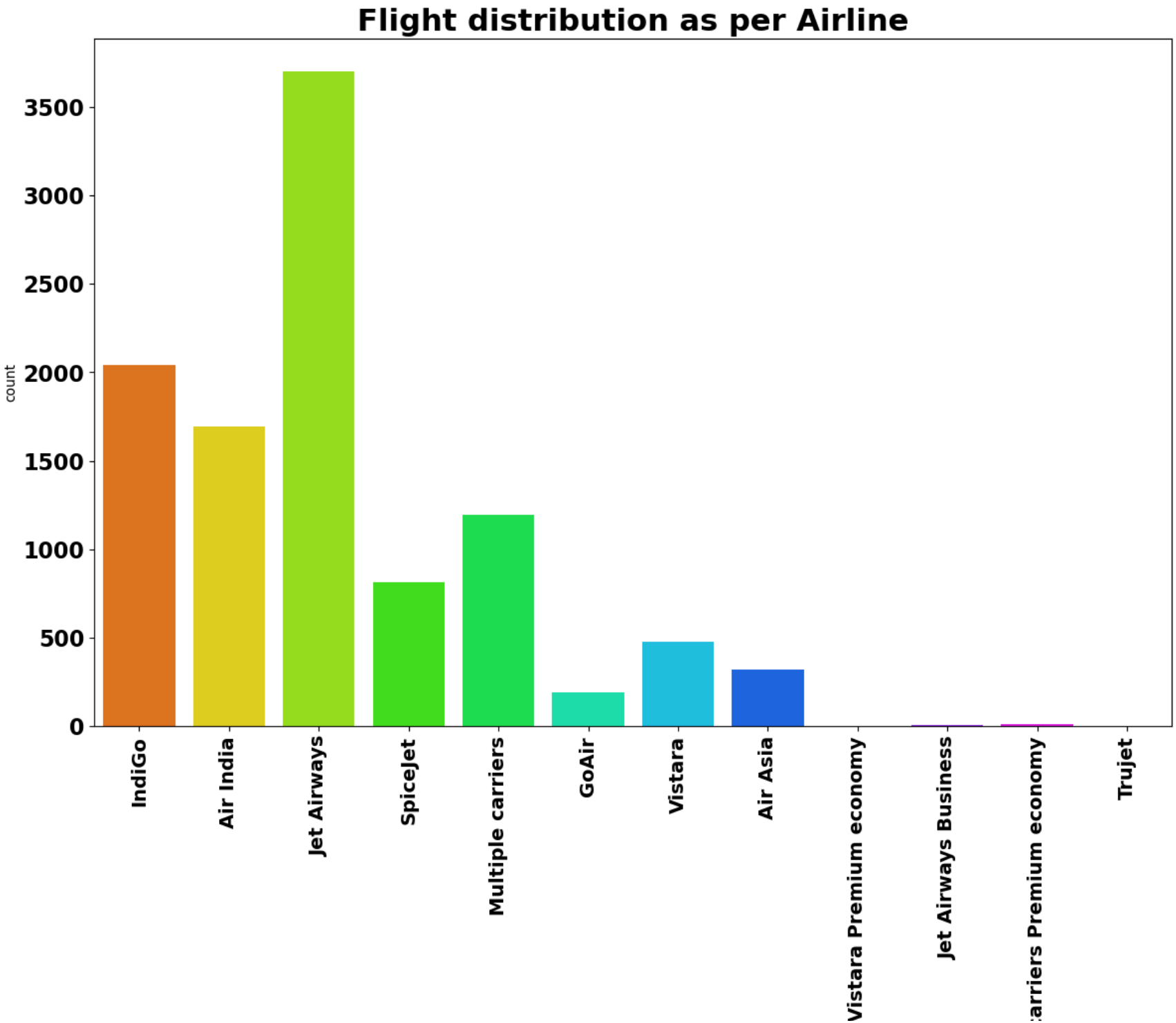
In [27]:
```
df.head()
```

Out[27]:

	Airline	Source	Destination	Route	Duration	Total_Stops	Additional_Info	Price	Journey_Day	Journey_Month	Dep_Hour	Dep_Min
0	IndiGo	Banglore	Delhi	BLR → DEL	170	non-stop	No Info	3897	24	3	22	20
1	Air India	Kolkata	Banglore	CCU → IXR → BBI → BLR	445	2 stops	No Info	7662	1	5	5	50
2	Jet Airways	Delhi	Cochin	DEL → LKO → BOM → COK	1140	2 stops	No Info	13882	9	6	9	25
3	IndiGo	Kolkata	Banglore	CCU → NAG → BLR	325	1 stop	No Info	6218	12	5	18	5
4	IndiGo	Banglore	Delhi	BLR → NAG → DEL	285	1 stop	No Info	13302	1	3	16	50

In [28]:

```
plt.figure(figsize=(14,9))
sns.countplot(x=df['Airline'],  palette='hsv')
plt.title('Flight distribution as per Airline', fontsize=22, fontweight='bold')
plt.xlabel('Airline', fontsize=18,fontweight='bold')
plt.xticks(fontsize=14,fontweight ='bold',rotation=90)
plt.yticks(fontsize=16,fontweight ='bold')
plt.show()
```


Flight distribution as per Airline
count
3500
3000
2500
2000
1500
1000
500
0
IndiGo
Air India
Jet Airways
SpiceJet
Multiple carriers
GoAir
Vistara
Air Asia
Vistara Premium economy
Jet Airways Business
Trujet

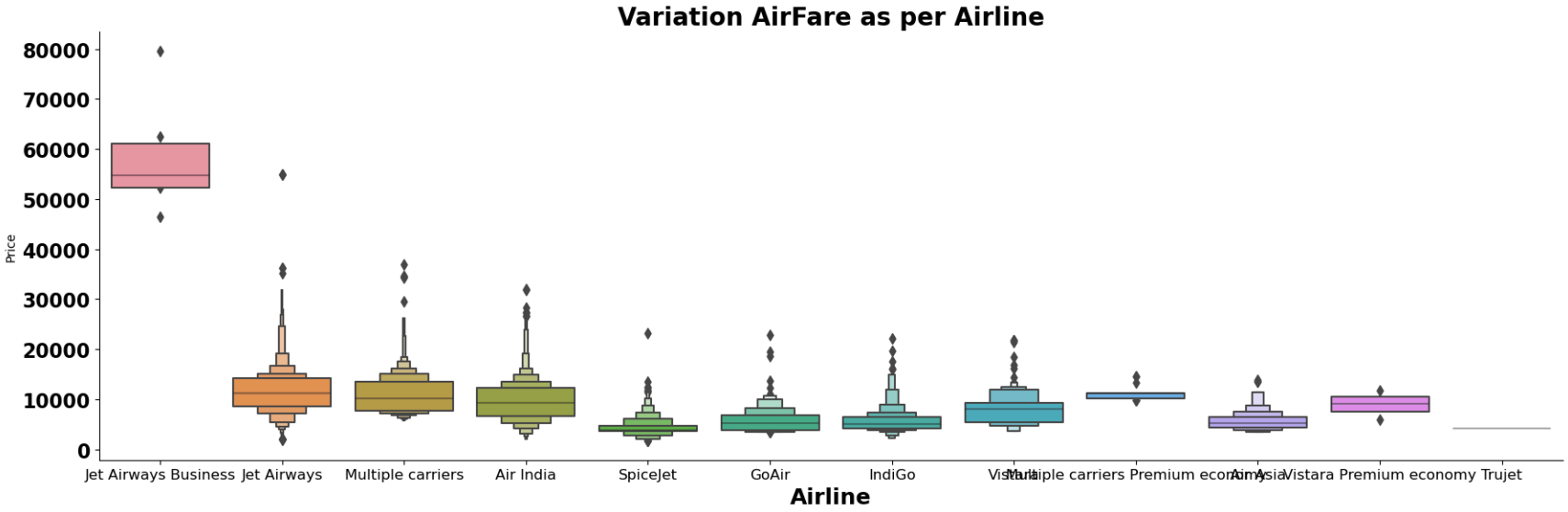
Multiple c

Airline

Comment :

- ***jet airways Airline runs highest number of flights out of all flights.***
- ***Very Few Premium class flights. #### Let Visualise Price according to these Airline***

In [29]:
```
# Airline vs AveragePrice
sns.catplot(y='Price',x='Airline',data= df.sort_values('Price',ascending=False),
            kind="boxen",height=6, aspect=3)
plt.title("Variation AirFare as per Airline",fontsize=20, fontweight='bold')
plt.xlabel('Airline', fontsize=18,fontweight='bold')
plt.xticks(fontsize=12)
plt.yticks(fontsize=16,fontweight ='bold')
plt.tight_layout()
plt.show()
```

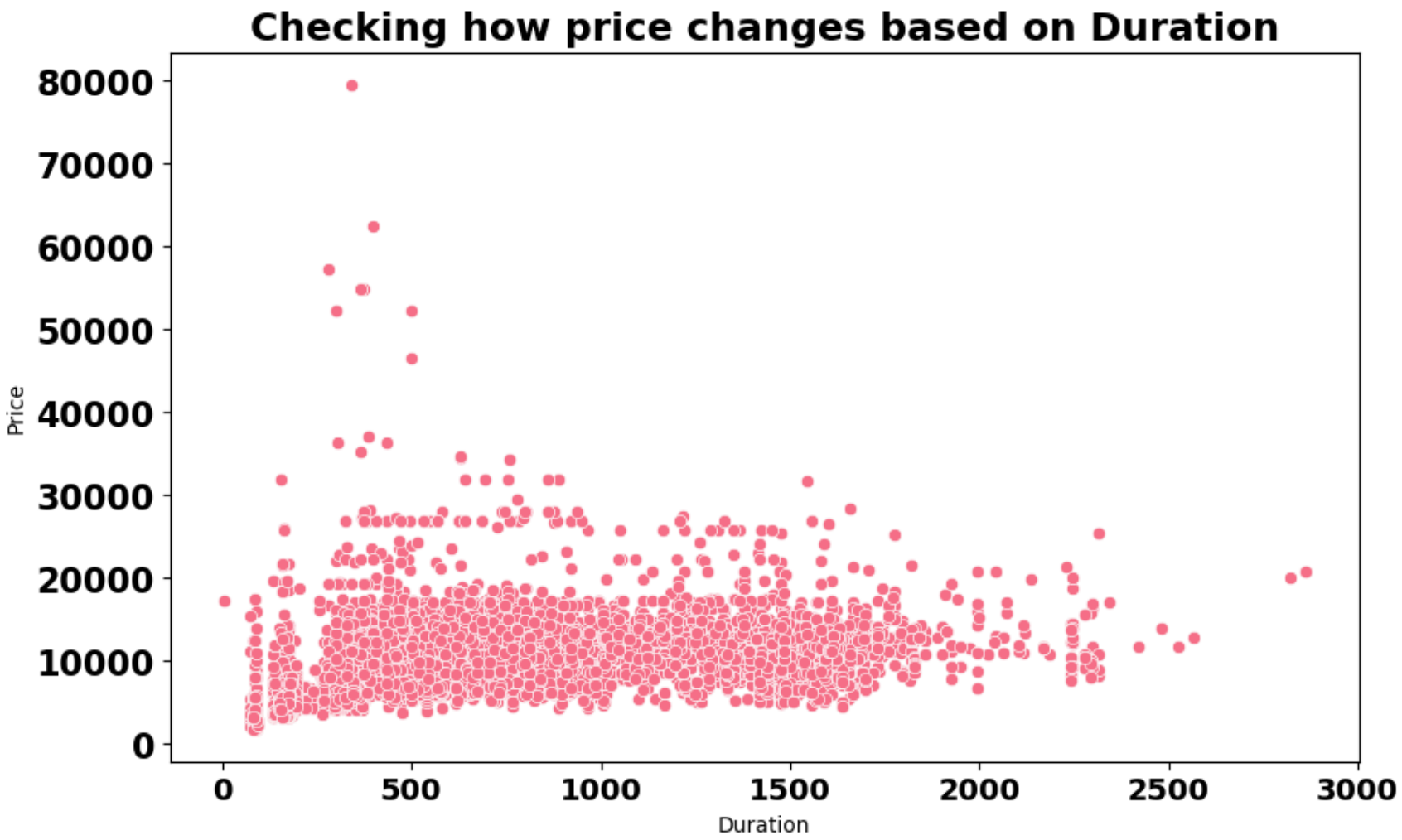


Observation:

- ***Full service airlines Jet airways and Air India are always highly priced due to various amenities they provide.***
- ***Low-cost carriers like indigo and spicejet have a lower and similar fare range***

Duration VS Price

In [30]:
```
#duration v/s AveragePrice
plt.figure(figsize=(10,6))
sns.scatterplot(data=df, x='Duration', y='Price')
plt.title("Checking how price changes based on Duration",fontsize=18, fontweight='bold')
plt.xticks(fontsize=14,fontweight ='bold')
plt.yticks(fontsize=16,fontweight ='bold')
plt.show()
```

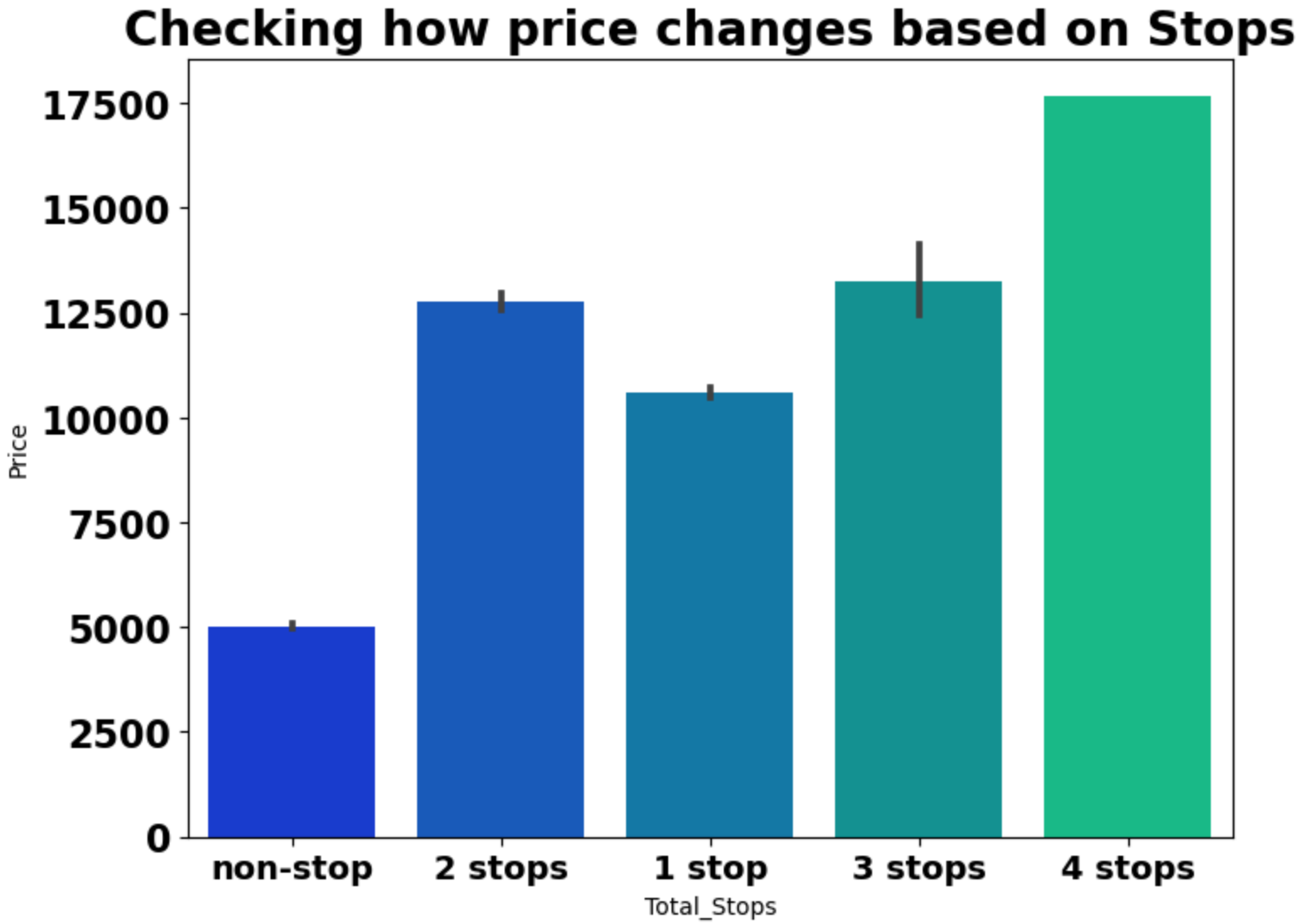


We know that duration(or distance) plays a major role in affecting air ticket prices but we see no such pattern here, as there must be there are other significant factors affecting air fare like type of airline, destination of flight, date of journey of flight(higher if collides with a public holiday)

Total Stops VS Price

In [31]:
```
plt.figure(figsize=(8,6))
sns.barplot(x=df["Total_Stops"],y=df["Price"],data=df,palette="winter")
plt.title("Checking how price changes based on Stops",fontsize=20, fontweight='bold')
plt.xticks(fontsize=14,fontweight ='bold')
```

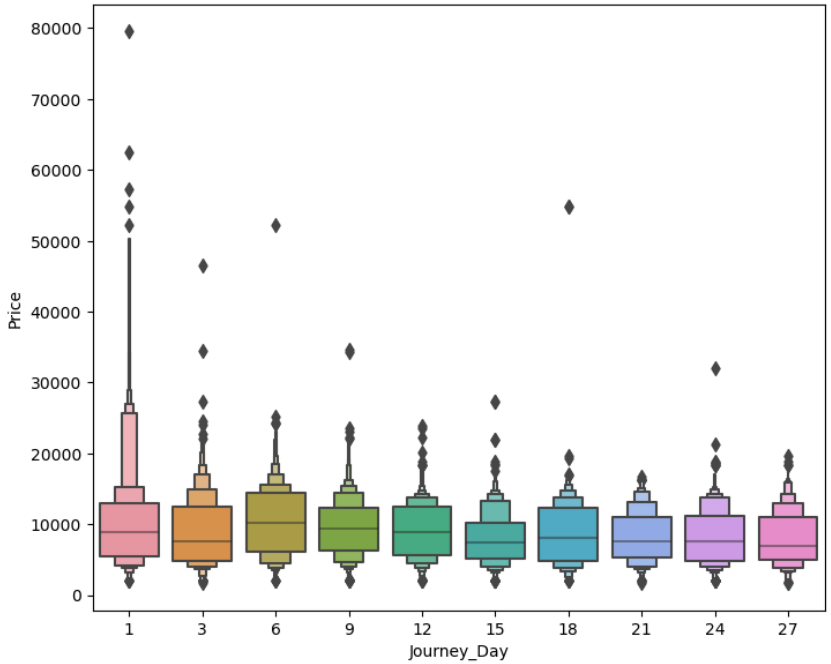
```
plt.yticks(fontsize=16,fontweight ='bold')
plt.show()
```

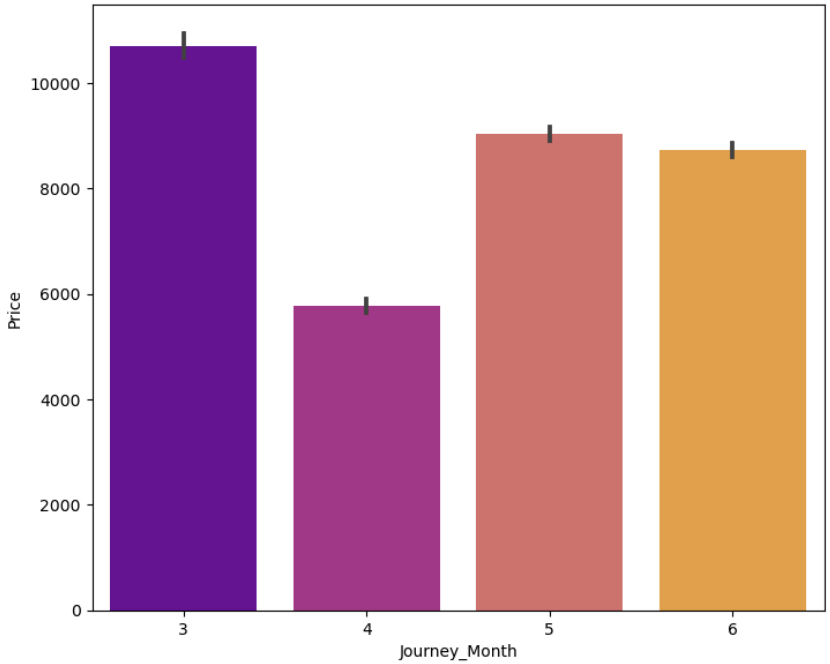


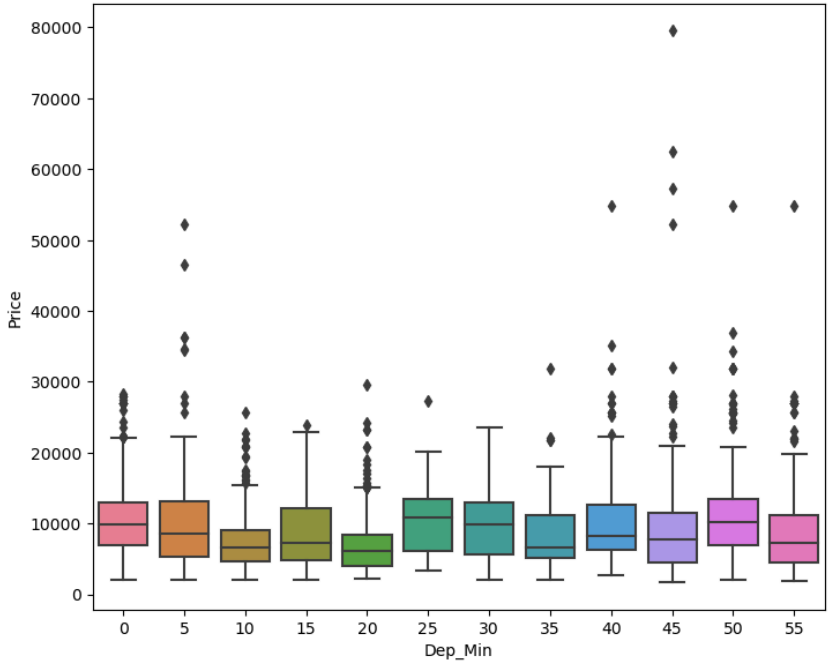
As a direct/non-stop flight is accounting for fare of only one flight for a trip, its average fair is the least. As the no. of stops/layovers increase, the fare price goes up accounting for no. of flights and due to other resources being used up for the same.

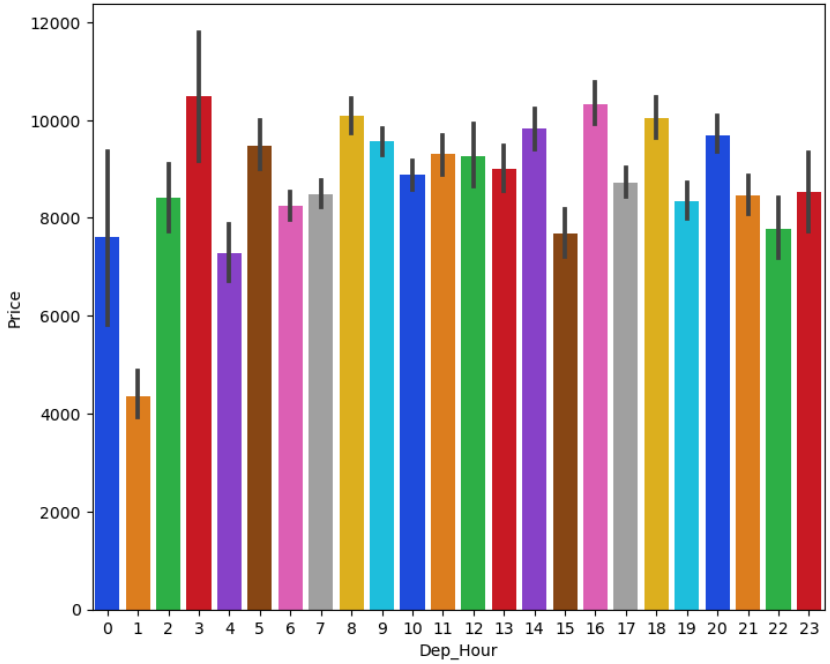
In [32]:
```
fig, axes = plt.subplots(2,2,figsize=(18,15))
```

```
# Checking relation between Journey_Day and Price
sns.boxenplot(x='Journey_Day',y='Price',ax = axes[0,0],data=df)

# Checking relation between Journey_Mionth and Price
sns.barplot(x='Journey_Month',y='Price',ax = axes[0,1],data=df,palette='plasma')

# Checking relation between Dep_Min and Price
sns.boxplot(x='Dep_Min',y='Price',ax=axes[1,0],data=df,palette='husl')

# Checking relation between Dep_Hour and Price
sns.barplot(x='Dep_Hour',y='Price',ax=axes[1,1],data=df,palette="bright")
plt.show()
```


Price
Journey_Day
1
3
6
9
12
15
18
21
24
27


Price
Journey_Month
3
4
5
6


Price
Dep_Min


Price
Dep_Hour

Observation :

- ***Airfare is high on Day 3 followed by Day 18.***
- ***January month are most expensive than others while airfare least expensive in April month.***

Encoding categorical data

In [20]:
```
Categorical=df.select_dtypes(include="object")
```

In [21]:
```
# Using Label Encoder on categorical variable
Categorical=df.select_dtypes(include="object")
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
for i in Categorical:
    df[i] = le.fit_transform(df[i])
df.head()
```

Out[21]:

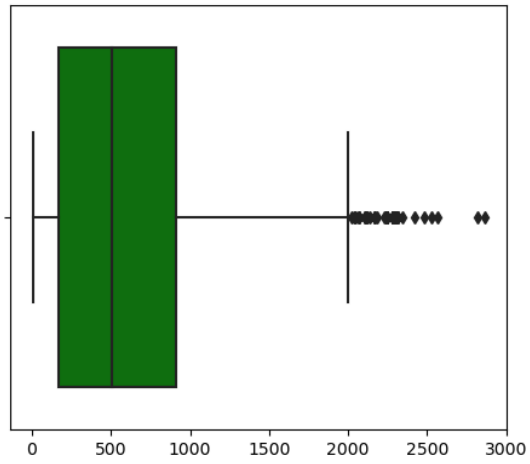
	Airline	Source	Destination	Route	Duration	Total_Stops	Additional_Info	Price	Journey_Day	Journey_Month	Dep_Hour	Dep_Min	Arri
0	3	0	2	18	170	4	6	3897	24	3	22	20	
1	1	3	0	84	445	1	6	7662	1	5	5	50	
2	4	2	1	118	1140	1	6	13882	9	6	9	25	
3	3	3	0	91	325	0	6	6218	12	5	18	5	
4	3	0	2	29	285	0	6	13302	1	3	16	50	

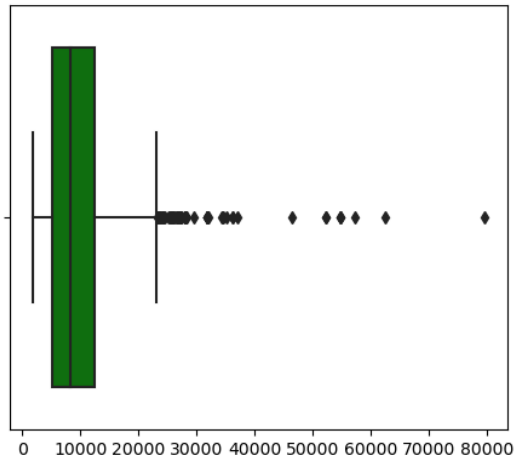
Feature selection and Engineering

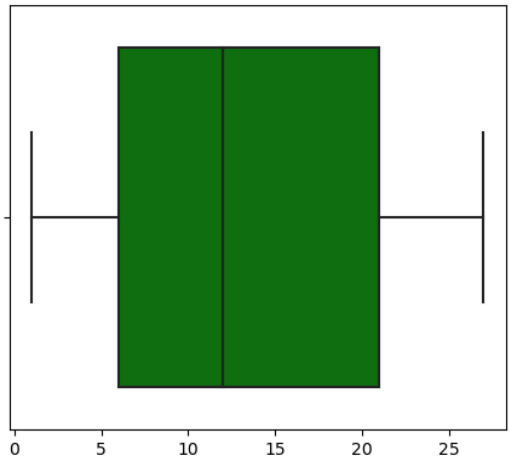
1. Outliers Detection and Removal

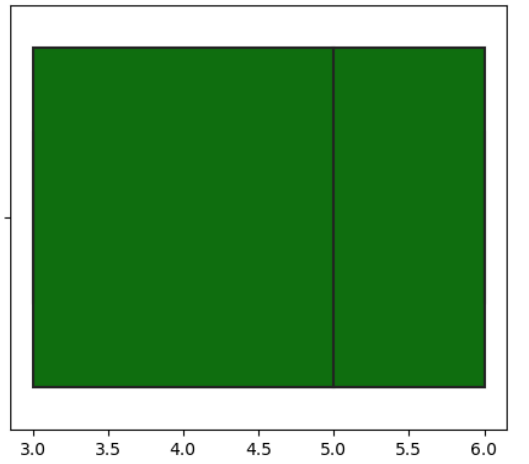
In [22]:
```
plt.figure(figsize=(18,10),facecolor='white')
plotnumber=1
```

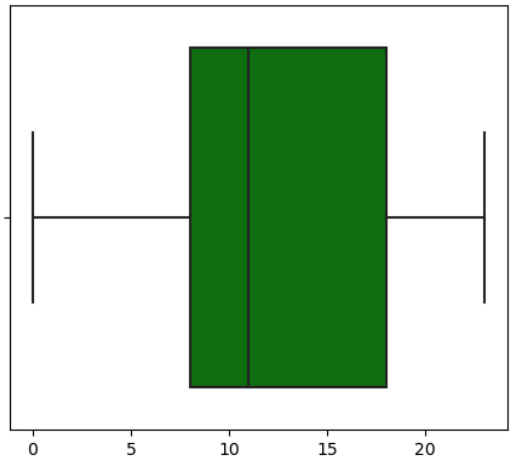
```
for i in Numerical:
    if plotnumber<=5:
        ax=plt.subplot(2,3,plotnumber)
        sns.boxplot(x=df[i],color='g')
        plt.xlabel(i,fontsize=20)
    plotnumber+=1
plt.show()
```











From Boxplot we can see outliers exist dataset.

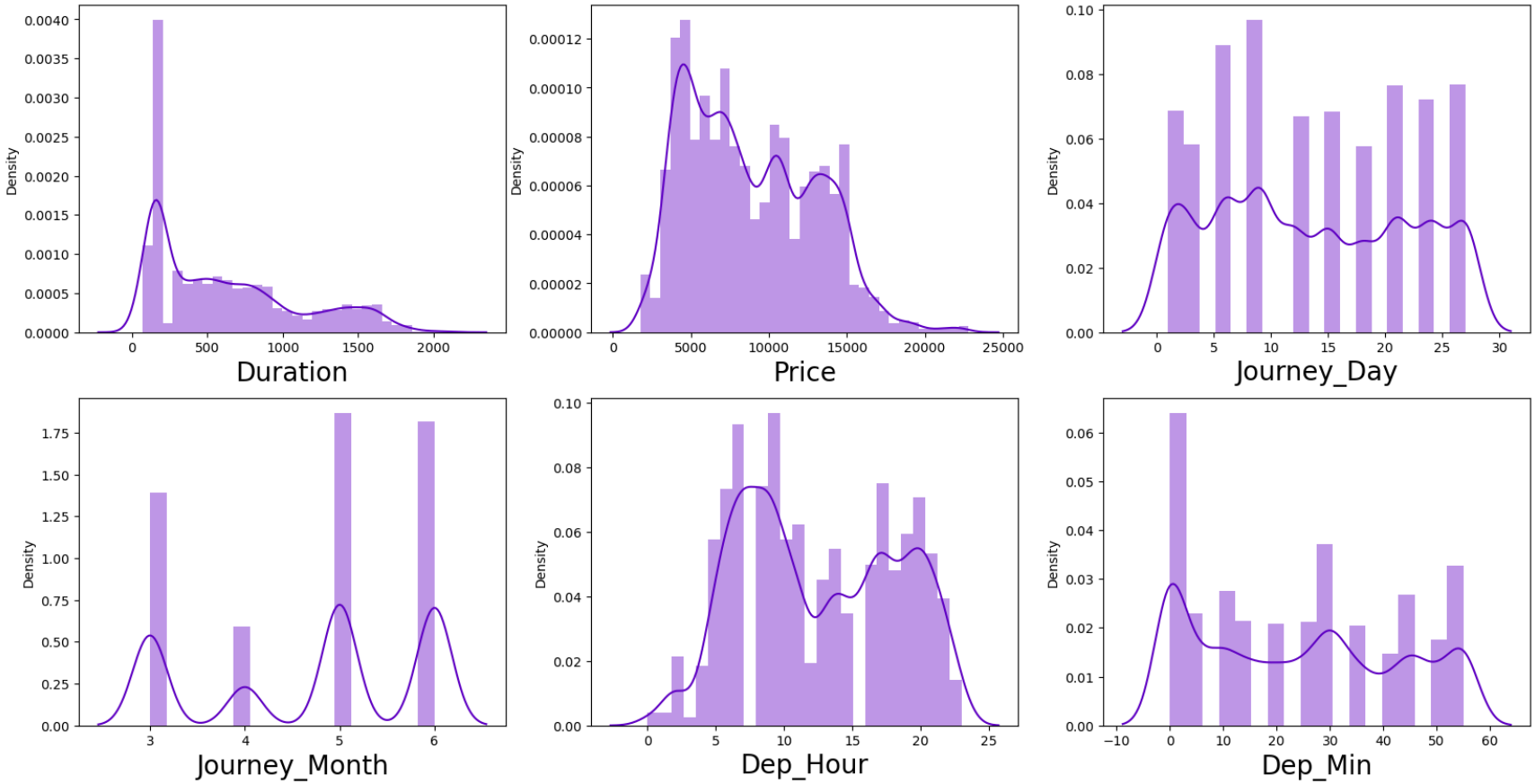
Outliers removal using Zscore method

In [23]:
```
from scipy.stats import zscore
z = np.abs(zscore(df))
df1 = df[(z<3).all(axis = 1)]

print ("Shape of the dataframe before removing outliers: ", df.shape)
print ("Shape of the dataframe after removing outliers: ", df1.shape)
print ("Percentage of data loss post outlier removal: ", (df.shape[0]-df1.shape[0])/df.shape[0]*100)

df=df1.copy() # reassigning the changed dataframe name to our original dataframe name
```

```
Shape of the dataframe before removing outliers:  (10461, 14)
Shape of the dataframe after removing outliers:  (10289, 14)
Percentage of data loss post outlier removal:  1.6442022751171015
```

2. Skewness of features

In [24]:
```
plt.figure(figsize=(20,10),facecolor='white')
sns.set_palette('gnuplot')
plotnum=1
for col in Numerical:
    if plotnum<=6:
        plt.subplot(2,3,plotnum)
        sns.distplot(df[col])
        plt.xlabel(col,fontsize=20)
    plotnum+=1
plt.show()
```



Skewness is important feature for continous data. There is no relevence of skweness for discrete numerical feature like month and categorical feature.So we gone ignore skewness present in discrete numerical and categorical feature.

We also going to ignore sknewness in target feature.

In [25]:
```
df.skew()
```

Out[25]:
```
Airline           0.721810
Source           -0.436014
Destination       0.850689
Route            -0.487883
Duration          0.820318
Total_Stops       0.599846
Additional_Info  -0.726010
Price             0.442553
Journey_Day       0.114526
Journey_Month    -0.409366
Dep_Hour          0.099829
Dep_Min           0.176249
Arrival_Hour     -0.371973
Arrival_Min       0.107114
dtype: float64
```

Price and duration are continous numerical data with skewed nature. Out of which Price is target variable, so ingore it.

We gone transform duration. For rest others we gone ignore skewness present as they discrete numerical and categorical feature

In [26]:
```
df['Duration'] = np.log1p(df['Duration'])
```

Checking skewness after transformation.

In [27]:
```
df['Duration'].skew()
```

Out[27]:
```
-0.14885157403409577
```

Standard Scaling

In [32]:
```
# Splitting data in target and dependent feature
X = df.drop(['Price'], axis =1)  #DataFrame
y = df['Price']  #Series
```

In [33]:
```
from sklearn.preprocessing import StandardScaler
scaler= StandardScaler()
X_scale = scaler.fit_transform(X)  #mean-0 std=1

#np.round(pd.DataFrame(X_scale,columns=X.columns).describe(),2)
```

In [34]:
```
from sklearn.model_selection import train_test_split
```

In [35]:
```
X_train,X_test,y_train,y_test=train_test_split(X_scale,y,test_size=0.2,random_state=42)
```

In [37]:
```
from sklearn.linear_model import LinearRegression
lr=LinearRegression()
lr.fit(X_train,y_train)
y_pred=lr.predict(X_test)
```

In [38]:
```
result = pd.DataFrame(np.c_[y_test,y_pred], columns = ["Original Price","Predict Predicted"])
result.head()
```

Out[38]:

	Original Price	Predict Predicted
0	7740.0	10439.137412
1	10368.0	10716.514586
2	10262.0	11024.653966
3	2647.0	3812.806706
4	6216.0	5518.570327

In [39]:
```
from sklearn.metrics import mean_squared_error, mean_absolute_error
```

In [42]:
```
print('Mean absolute error :', mean_absolute_error(y_test,y_pred))
print('Mean squared error :', mean_squared_error(y_test,y_pred))
print('Root Mean Squared Error:', np.sqrt(mean_squared_error(y_test,y_pred)))
print('\n')
from sklearn.metrics import r2_score
print(' R2 Score :')
print(r2_score(y_test,y_pred))
```

```
Mean absolute error : 2272.1194335422656
Mean squared error : 8411868.962797116
Root Mean Squared Error: 2900.3222170643585


 R2 Score :
0.49818936877220676
```

In [ ]:
```
```